



DISASTER TWEET CLASSIFICATION

Presented by:

Annelize Krause



The background is a dense, overlapping collage of various digital content. It includes numerous smartphone screens displaying different apps, social media feeds, and images. Some screens show a 'Login' button, a calculator, a video of a person, a 'LOL!' speech bubble, a night scene with a bridge, and a person's face. The overall theme is digital communication and information overload.

THE PROBLEM

Thousands of social media posts per minute. Some report real disasters, but most don't. Can I build a classification model to tell them apart?



WHY IT MATTERS

Correctly identifying disaster posts in real time could decrease emergency response times and create better outcomes.

THE DATASET

Use Kaggle dataset of 7,613 pre-labelled tweets to train and evaluate a prediction model.

Binary target:

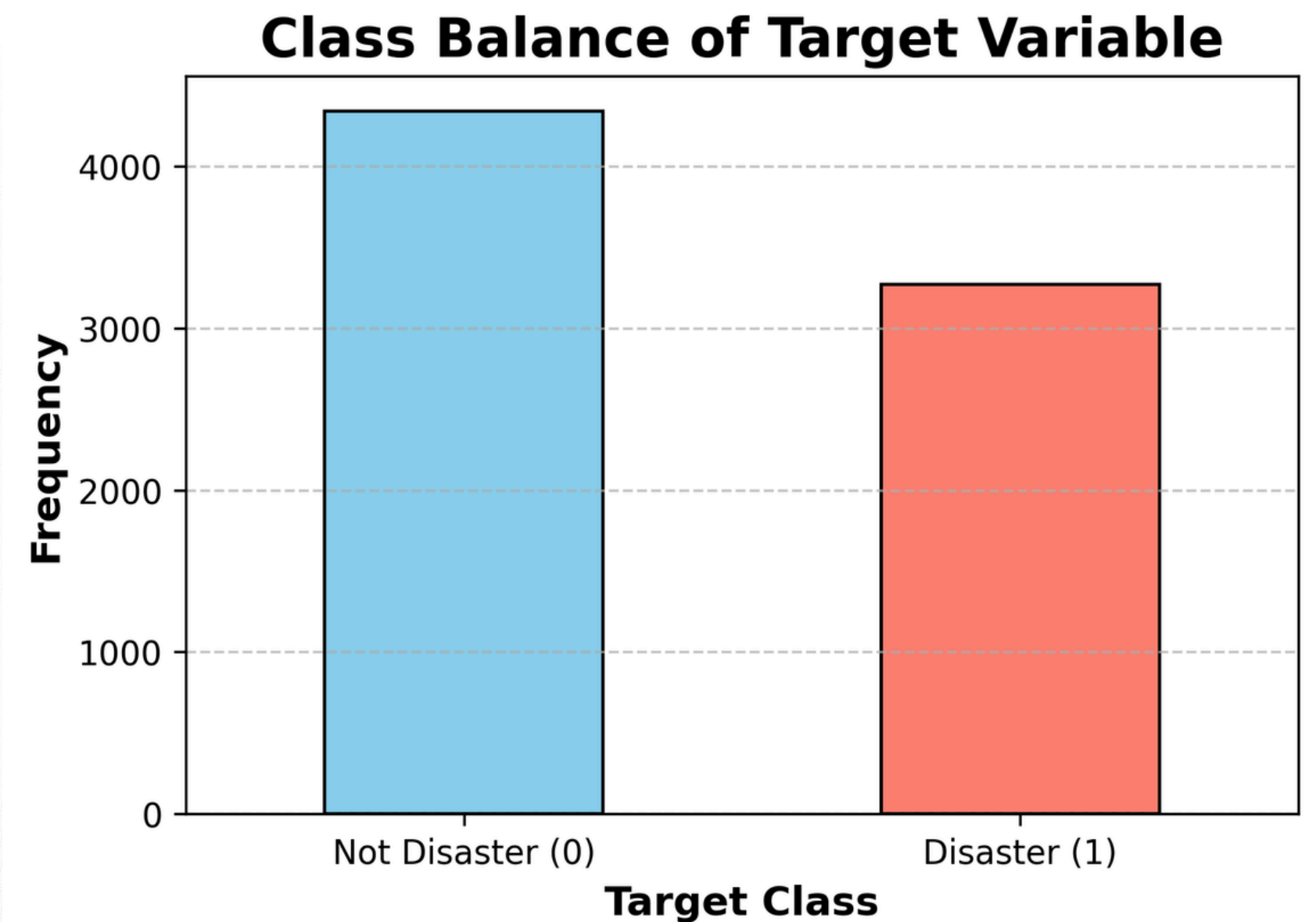
0 = Not Disaster

1 = Disaster

Distribution:

57% = Not disaster

43% = Disaster



TEXT PREPROCESSING

Tweets are noisy and preprocessing removes noise while preserving meaning:

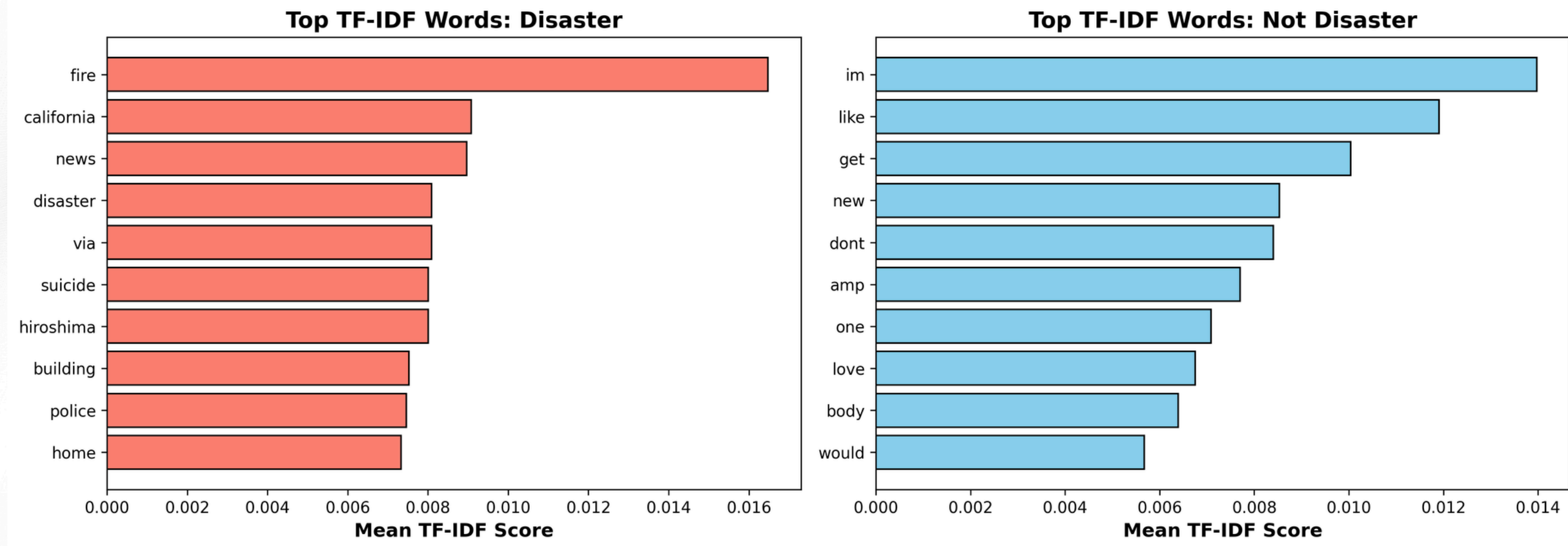
- Lowercase all text
- Removed URLs, @mentions, special characters, numbers
- Kept hashtag words (removed `#` symbol only)
- Removed stop words using NLTK
- Lemmatized words to their base form



FEATURE EXTRACTION

TF-IDF converts text into numbers a model can learn from by:

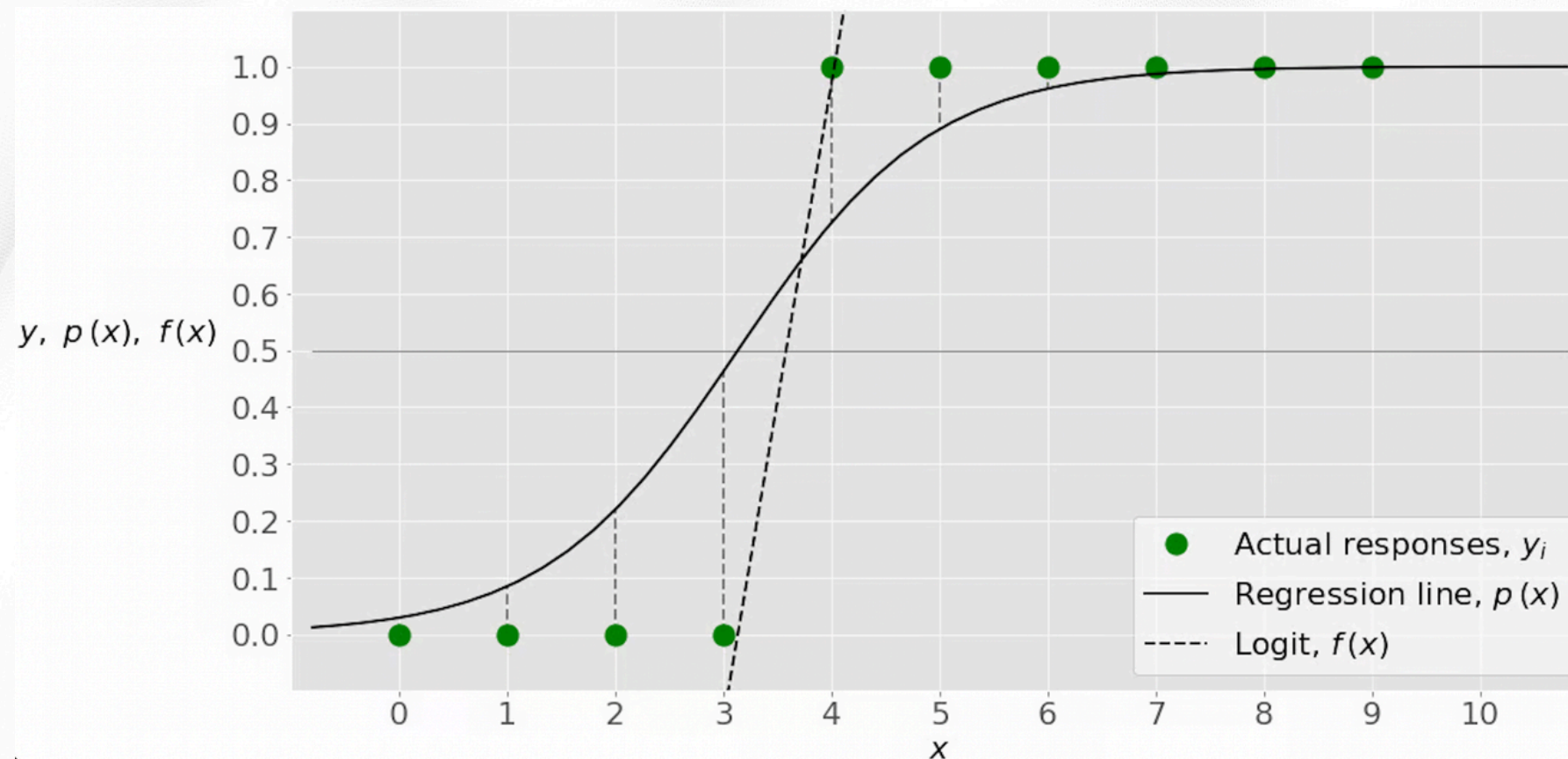
- scoring words by how important they are to a specific tweet relative to the whole dataset
- common words get low scores, distinctive words get high scores
- result: 11,708 features from the training vocabulary



MODEL SELECTION

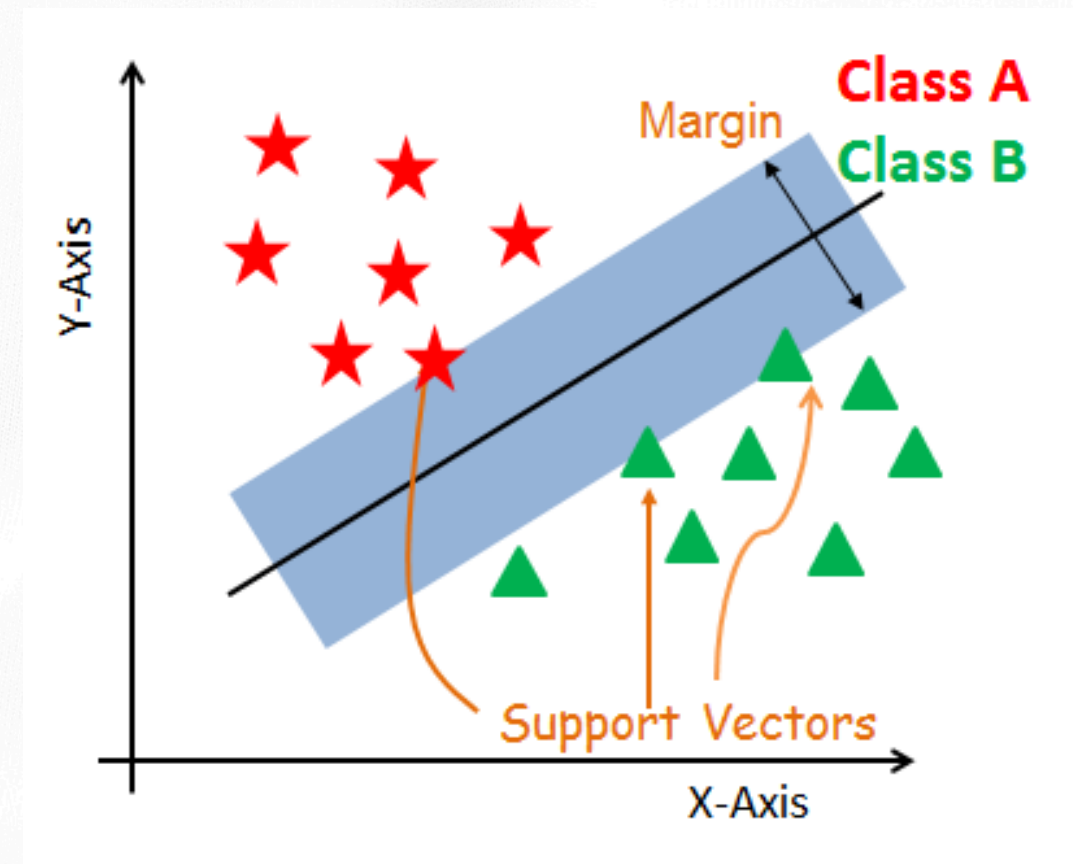
Logistic Regression

- Uses a sigmoid function to estimate probability of each class
- Well suited for binary classification (disaster / not disaster)
- Works well with high-dimensional sparse data like TF-IDF vectors
- Outputs probabilities, making predictions easy to interpret



Linear SVC

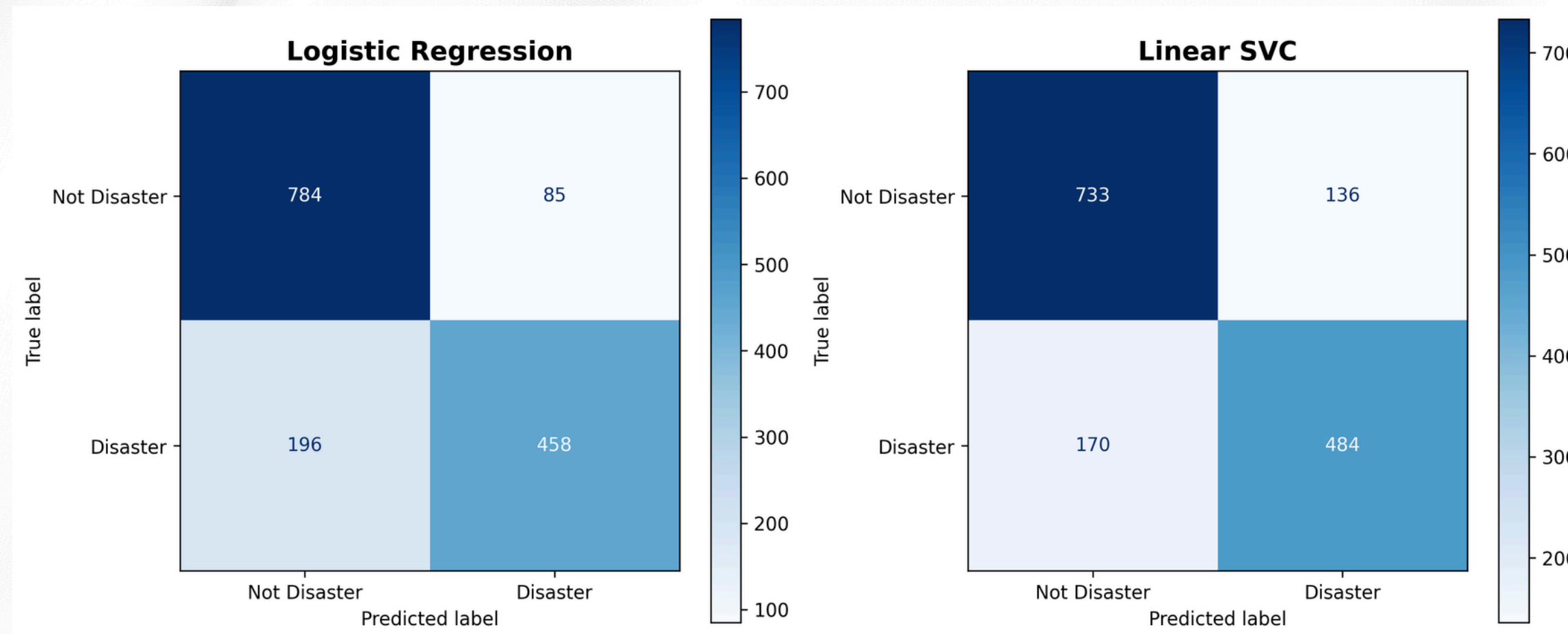
- Finds decision boundary that maximises margin between classes
- Strong performance on text classification tasks
- Different approach to same problem helps validate results



BASELINE RESULTS

Both models reach ~80-82% accuracy, but the confusion matrices reveal a precision-recall trade-off:

- **Logistic Regression** is more precise 84% with lower recall 70%.
- **Linear SVC** is less precise 78% catching more actual disasters 74%.





HYPERPARAMETER TUNING

- Can we improve the 30% miss rate by adjusting the model's settings?
- Used GridSearchCV to test 24 combinations across vocabulary size, word groupings, and regularisation strength
- Modest improvement, meaning the baseline was already near optimal for this model type.

Metric	Baseline	Tuned
Accuracy	82%	83%
Disaster Recall	0.70	0.72
Disaster F1	0.77	0.78



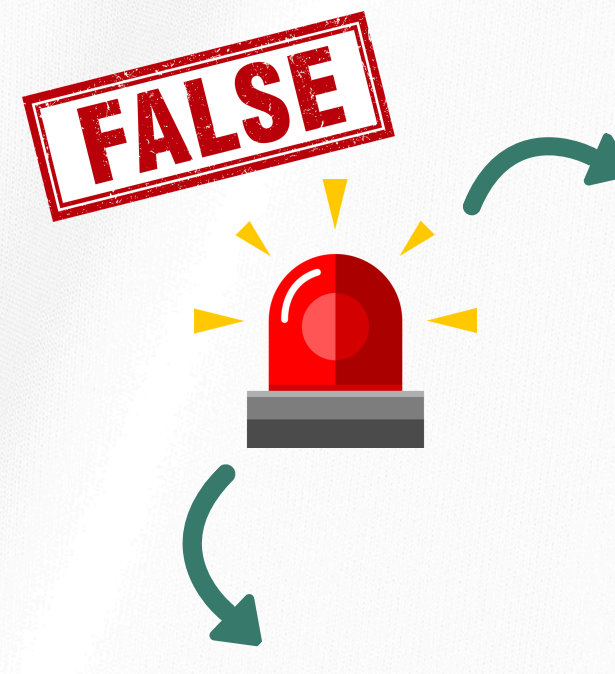
HITS & MISSES

What works well:

- Explicit disaster keywords
- Factual, news-like language

What does it fail on:

- Figurative language
- Sarcasm and slang
- Short or vague tweets with little context

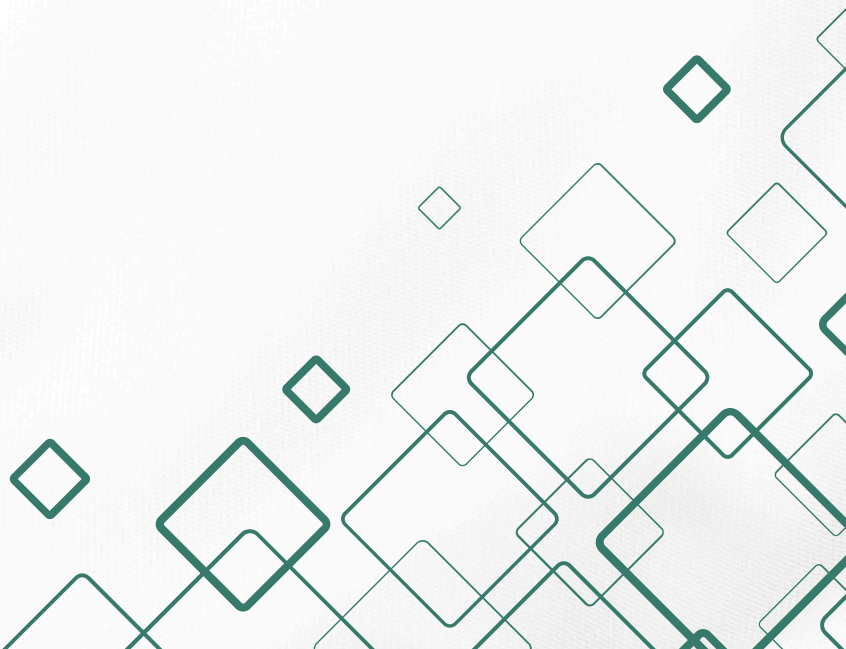




KEY TAKEAWAYS

- A simple Logistic Regression + TF-IDF pipeline achieves 83% accuracy on disaster tweet classification
- Text preprocessing choices have real trade-offs (removing stop words can lose negation)
- Precision vs recall trade-off depends on the application — for disaster detection, recall should be prioritised
- The main limitation is the bag-of-words representation, which cannot capture context or figurative language





LIMITATIONS & IMPROVEMENTS

Limitations:

- TF-IDF ignores word order and context
- Small dataset (7,613 samples)
- Class imbalance affects disaster recall
- Preprocessing removes potentially meaningful signals (stop words, punctuation)

Future improvements:

- Word embeddings (Word2Vec, GloVe) for semantic features
 - Transformer models (BERT, BERTweet) for context-aware classification
 - Class balancing techniques (SMOTE, class weighting)
- 



THANK YOU FOR WATCHING!

Connect with me:

-  annelizekrause.com
-  linkedin.com/in/annelizekrause
-  github.com/krauseannelize
-  krauseannelize@gmail.com

